

# Programming is an engineering profession

This chapter resulted from a collection of material from various occasional articles. It was put together as a sort of prospectus for an M.Sc. course at Oxford University which aimed to attract programmers from industry. The text of this chapter is a version of the PRG-27 Oxford monograph published in May 1982; a shorter version of the paper is available in [83]; the same text is published under a different title in [89].

In earlier times and in less advanced societies, the welfare of a community depended heavily on the skill and dedication of its craftsmen – the millers, blacksmiths, spinners, weavers, joiners, thatchers, cobblers and tailors. A craftsman possesses skills, not shared by his clients, which he has acquired by long and ill-paid apprenticeship to a master of his craft. He learns exclusively by imitation, practice, experience and by trial and error. He knows nothing of the scientific basis of his techniques, nothing of geometry or even of drawing, nothing of mathematics, or even of arithmetic. He cannot explain how or why he does what he does, and yet he works effectively, by himself or in a small team, and can usually complete the tasks he undertakes in a predictable time scale and at a fixed cost, and with results that are predictably satisfactory to his clients.

The programmer of today shares many of the attributes of the craftsman. He learns his craft by a short but highly paid apprenticeship in an existing programming team, engaged in some on-going project; and he develops his skills by experience rather than by reading books or journals. He knows nothing of the logical and mathematical foundations of his profession. He does not like to explain or document his activities. Yet he works effectively, by himself or in small teams, and he sometimes manages to complete the

---

C. A. R. Hoare, Programming is an engineering profession. In P. J. L. Wallis (ed.), *Software Engineering*, State of the Art Report Volume 11, No. 3, pp. 77–84. Reprinted with permission. Copyright © 1983, Pergamon/Infotech.

tasks he undertakes at the predicted time within the predicted costs, and to the satisfaction of his client.

In primitive societies of long ago we hear of another class of specialist on whom the welfare of the community depended. Like the craftsman, he is dedicated to his task; like the craftsman he is regarded with respect, perhaps even tinged with awe, by his many satisfied clients. There are several names given to such a man – a seer, a soothsayer, a sorcerer or wizard, a witch doctor or high priest. I shall just call him a high priest.

There are many differences between the craftsman and the high priest. One of the most striking is that the high priest is the custodian of a weighty set of sacred books, or magician's manuals, which he alone is capable of reading. When he is consulted by his client with some new problem, he refers to his sacred books to see whether he can find some spell or incantation which has proved efficacious in the past; and having found it, he tells his client to copy it carefully and use it in accordance with a set of elaborate instructions. If the slightest mistake is made in copying or in following the instructions, the spell may turn to a curse, and bring misfortune to the client. The client has no hope of understanding the nature of the error or why it has evoked the wrath of his deity – the high priest himself has no inner understanding of the ways of his god. The best the client can hope for is to go right back to the beginning, and start the spell again; and if this does not work, he goes back to the high priest for a new spell.

And that is another feature of the priesthood – when something goes wrong, as it quite often does, it somehow always turns out to be the ignorance or stupidity or impurity or wickedness of the client; it is never the fault of the high priest or his god. It is notable that when the harvest fails, it is the high priest who sacrifices the king, never the other way round.

Programmers of the present day share many of the attributes of the high priest. We have many names – coder, systems analyst, computer scientist, informatician, chief programmer: I shall just use the word 'programmer' to stand for them all. Our altars are hidden from the profane, each in its own superbly air-conditioned holy of holies, ministered to night and day by a devoted team of acolytes, and regarded by the general public with mixed feelings of fear and awe, appropriate for their condition of powerless dependence.

An even more striking analogy is the increasing dominance of our sacred books – the basic software manuals for our languages and operating systems which have become essential to our every approach to the computer. Only 30 years ago our computers' valves and tanks and wires filled the walls and shelves of a large room, which the programmer would enter, carrying in his pocket his programming manual – a piece of folded cardboard known as the 'facts card'. Now the situation is reversed: the programmer enters a large room whose walls and shelves are filled with

software manuals: but in case he wants some urgent calculations he carries in his pocket – a computer.

### 18.1 The rise of engineering

In recent centuries with the advance of technology, we have seen the emergence of a new class of specialist – the professional engineer. The most striking characteristic of an engineer is the manner in which he qualifies for entry into his profession: not only does he work out the long apprenticeship of the craftsman, not only does he undergo the brief graduation of initiation ceremonies of the high priest, but both of these are preceded by many years of formal study in schools and in universities. His education covers a wide range of topics, including the mathematical foundations of the differential calculus, the derivation and solution of complex equations, the physical principles underlying the science of materials, as well as the specific technicalities of a particular branch of his subject, and a large catalogue of known design methods and specific practical techniques. However, this is only a start: during his professional career the engineer will expect to continue his education, to expand his skills, and to keep pace with technological progress by continued study of new books and learned journals, and attendance of specialist orientation courses. The older craftsmen will complain that the engineer already knows far more than he needs for the day-to-day practice of his profession; but his colleagues and clients will realize that the weight of background learning develops his good judgement and increases his competence and authority at all times; even if a recondite scrap of knowledge is used only once in his career, then the learning has paid for itself many times over.

We would like to claim that computer programming has transcended its origins as a craft, has avoided the temptation to form itself into a priesthood, and can now be regarded as a fully fledged engineering profession. Certainly, we have some right to this claim. Through our professional Societies we have formulated a code of professional ethics and a structure and syllabus of professional examinations. We discharge our duty to the community by giving evidence to government commissions on social consequences of computing, on privacy, on employment. Because of the great demand for our services, our clients and employers are willing to offer us professional salaries, and it is hardly likely we shall refuse them.

But more than this is needed for true professional status. What is the great body of professional knowledge common to all educated programmers? Where are the reference libraries of standard works on known general methods and specific techniques and algorithms oriented to particular applications and requirements? What are the theoretical,

mathematical or physical principles which underlie the daily practice of the programmer? Until recently, these questions had no answer. Now the answers are beginning to emerge. We can point to the ACM curriculum for the study of computer science at university as a corpus of common knowledge for the programmer, though the proportion of computer science graduates in the programming profession is still low. Don Knuth's books (1973) on the art of computer programming form an excellent encyclopaedia of known techniques – but only three volumes have so far appeared, and how many programmers consult even those? And finally, we have only recently come to a realization of the mathematical and logical basis of computer programming: we can now begin to construct program specifications with the same accuracy as an engineer can survey a site for a bridge or road, and on this basis we can now construct programs proved to meet their specification with as much certainty as the engineer assures us his bridge will not fall down. Introduction of these techniques promises to transform the arcane and error-prone craft of computer programming to meet the highest standards of a modern engineering profession.

Let me expand on the nature and consequences of this discovery. It is like the Greek discovery of axiomatic geometry as the basis of the measurement of land, map-making, and its later use in plans and elevations for the design and construction of buildings and bridges. It is like the discovery of the Newtonian laws of motion and the differential calculus as the basis of astronomy as well as more mundane tasks like the navigation of ships and the direction of artillery fire. It is like the discovery of stress analysis as the basis for the reliable and economic construction of steel frame buildings, bridges, and oil platforms.

This is just analogy. The analogy between programming and more traditional engineering disciplines is a very fruitful one, and it has been the basis of many theoretical and practical advantages to our profession. But analogies are dangerous, especially if they are applied without consideration of the underlying reality. I would therefore like to concentrate attention on three points where the analogy breaks down:

- (1) The difference in methods by which we achieve reliability.
- (2) Our difficulty in establishing structural isolation of parts of a large program.
- (3) Our need for improved intellectual and mechanical tools.

## 18.2 Reliability

In principle, we should find it much easier than other professional engineers to achieve the highest standards of quality, accuracy and predictability of

time scale and cost because the raw materials with which we work are much simpler, more plentiful, and much more reliable. Our raw materials are the binary digits in the stores and registers, disks and tapes of our computers. Our problem is that we have too many of them rather than too few. These bits are manipulated exactly in accordance with our instructions, at a rate of millions of operations per second for many weeks or months without mistake. When the hardware does go wrong it is the engineer, not the programmer, who is called upon to mend it.

That is why computer programming should be the most reliable of all professional disciplines. We do not have to worry about problems of faulty castings, defective components, careless labourers, storms, earthquakes or other natural hazards: we are not concerned with friction or wear or metal fatigue. Our only problems are those we make for ourselves and our colleagues by our over-ambition or carelessness, by our failure to recognize the mathematical and theoretical foundations of programming, and our failure to base our professional practice upon them.

Yet in some ways the engineers have an advantage over us. Because they are dealing with continuously varying quantities like distance, temperature, and voltage, it is possible for them to increase confidence in the reliability of an engineering product by testing it at the extremes of its intended operating range, for example, by exposure to heat and cold, or by voltage margins. We do the same in program testing, but in our case it is futile. Firstly we have to deal with impossibly many more variables and secondly these variables take discrete values, for which interpolation and extrapolation are wholly invalid. The fact that a program works for value zero and value 65 535 gives no confidence that it will work for any of the values in between, unless this fact is proved by logical reasoning based on the very text of the program itself. But if this logical reasoning is correct, then there was no need for the test in the first place. That is why it is an essential prerequisite to the improvement of our professional practices that we learn to reason effectively about our programs, to prove their correctness before we write them, so that we know that they will not only pass all their tests, but will go on working correctly forever after.

### 18.3 Structure

Traditional engineers have a further advantage over programmers: when they split a complex design into a number of component parts, to be designed independently of each other, they can take advantage of the spatial separation of the parts to ensure that there can be no unexpected interaction effects. If the parts are wholly unconnected, this is very easy to check by simple visual inspection. Thus when we turn our car to the left, we may be

very confident that this will have no direct effect on the cigarette-lighter, the rear mirror, or the carburettor. When such interaction effects do occur, they are recognized as the most difficult to trace and eliminate.

But in the programming of conventional computers, there is no similar concept of spatial separation. Any instruction in a binary computer program can modify any location in the store of the computer including those that contain instructions. And if this happens incorrectly only once in a thousand million instructions executed, the consequences of the whole program will be totally unpredictable and uncontrollable. There is no hope that a prior visual inspection of the binary content of store will enable us to check that such interaction cannot occur, or to find the cause of its occurrence afterwards. There is no structure or isolation of components in a binary computer program, other than that which has been carefully designed into it from the start, and maintained by the most rigorous discipline throughout implementation.

In spite of this, the programmer is often asked to include some feature into his program as an afterthought: and the only quick way to do this is to insert new instructions which cross all the boundaries between the carefully isolated components and violate all the structural assumptions on which the original design was based. It would be repugnant to an engineer to introduce direct cross-coupling effects between the steering and carburettor of a motor car, or the tape decks and floating-point unit of a computer. A programmer is all too willing to do his best and his profession gets a bad name when unpredicted side-effects occur.

A partial solution to this problem lies in the use of a high-level language like ALGOL 60 with secure rules governing the scope, locality, and types of variable. In such a language the programmer can declare the structure of his program and data, stating which groups of variables are to be accessed or changed by which parts of his program. An automatic compiler can then check that the appropriate disciplines have been observed throughout the whole of a large program, and can therefore give the same confidence to the programmer as the engineer gains by spatial separation of his components. Further confidence can be gained by running the program on a machine like the Burroughs 5500 which makes similar checks while the program is running. In better established engineering disciplines, the observance of such elementary safety precautions has long been enforced by legislation. It is the law that dictates the measures that prevent unwanted interaction effects between an industrial machine and the body of its operator.

## **18.4 Tools of the trade**

This brings me to the final disadvantage suffered by the programmer, the poor quality of the tools of his trade. I refer to his programming languages,

operating systems, utility programs, library subroutines, all of which are supplied in profusion by the manufacturer of his computer. Many of these are so complicated that mastery of them absorbs all his intellectual efforts, leaving him little energy to apply to his client's original problem. Some operating systems are so poorly designed that they require 20 reissues (or 'releases') spread over a decade, before the original design faults have been rendered tolerable. They are so unreliable that each issue has a thousand faults corrected by the next issue, which introduces a thousand new faults of its own. When finally the agony of reissues comes to an end, instead of rejoicing, the poor programmer is cajoled or forced to accept an early issue of some 'new' product. Such complexity, unreliability, and instability of basic tools were doubtless endured by engineers of each newly emergent discipline: but gradually the engineers developed better toolkits for their own use. That is a task which still faces the programming profession today, the design of programming tools which are reliable, stable, convenient and, above all, simple to understand, control, and use.

A crude measure of the simplicity of an engineering tool is the length of the manual required to give a full and complete account of how to use it and avoid misusing it. At present our software manuals are both voluminous and inadequate. I believe that a solution to our problems can be sought in the design of software which can be completely described by shorter manuals. If an electronic engineer finds a method of satisfying with 20 components a need which has hitherto required 30, the value of his discovery is immediately recognized and is often highly rewarded by fame or by money. When a software engineer designs a product that can be fully defined in 20 pages of manual, when the rival product has been inadequately defined in 100, his achievement is just as great, and possibly more beneficial, for he has achieved an economy in our scarcest resource – not silicon or even gold, but our own precious human intellect.

### **18.5 How do we get there from here?**

My description of the professional achievement of programmers of the future may seem to be nothing but an academic dream – a pleasant one for our clients, but perhaps something more like a nightmare for us. However are we going to make such a fantastic improvement in our working methods? We are like the barber-surgeons of earlier ages, who prided themselves on the sharpness of their knives, and the speed with which they could dispatch their duties, either of shaving a beard or amputating a limb. Imagine the dismay with which they greeted the academic who told them that the practice of surgery should be based on a long and detailed study of human anatomy, on familiarity with surgical procedures pioneered by great doctors of the past and that it should be carried out only in a strictly

controlled 'bug-free' environment, far removed from the hair and dust of the normal barber's shop. Even if they accepted the validity and necessity for these improvements, how would they ever achieve them? How could all those hairdressers be re-educated in the essential foundations of surgery? Clearly, a two-week course in Structured Surgery is all that we can readily afford. But more is needed, much more.

First we need good books which can be studied by programmers and programming teams to familiarize themselves with the concepts of mathematical proof, and show how proof methods may be applied to the everyday practice of program specification, design, and implementation. Such books are beginning to appear in the publishers' lists, one of which is (Gries 1978).

We also need a journal in which practising programmers can read the results of current research to keep themselves up to date with the most effective technology. A new journal of this kind, entitled *The Science of Computer Programming*, has just been founded.

Most of the books and articles on programming methods are of necessity illustrated only by small examples. Indeed, many of the programming methods advocated by the authors have never yet been applied to large programs. This is not a defect of their research, it is necessity. All advances in engineering are tested first on small-scale models, in wave tanks, or in wind tunnels. Without models, the research would be prohibitively expensive, and progress would be correspondingly slow.

Nevertheless, I believe that the time has come to attempt to scale up the use of formal mathematical methods to industrial application. This can best be achieved by collaborative development projects between a university or polytechnic and an industrial company or software house. Such a project might be an entirely new program, or it might be a restructuring or redesign of some existing software product in current use, perhaps one which has lost its original structure as a result of constant amendment and enhancement. The great advantage of these joint projects is that they bring home to academic researchers some of the exigencies of working on much larger programs: and they give a practical training in formal methods to larger numbers of experienced programmers in industry. This is technology transfer in its best sense — a transfer of benefits in both directions.

## 18.6 Education

As I have emphasized already, the major factor in the wider propagation of professional methods is education, an education which conveys a broad and deep understanding of theoretical principles as well as their practical application, an education such as can be offered by our universities and



polytechnics. Lecturers and professors regard it as their duty and privilege to keep abreast with the latest developments in their subjects, and to adapt, improve and expand their courses to pass on their understanding to their students. Many entrants to computer science courses have acquired a familiarity with the basic mechanics of programming at their schools; and at university they are ready to absorb the underlying mathematical principles, which will help them to control the complexity of their designs and the reliability of their implementations.

Over the next decades, while the graduates of computer science courses are entering their profession, we will have an extremely awkward period, in which almost none of the senior professionals and managers will have any knowledge or understanding of the new methods, while those whom they recruit will seem to them to be talking academic gibberish. This could be a grave hindrance to the development of our profession. Furthermore, it would be a terrible wasted opportunity, because one of the major benefits of the technique of mathematical abstraction is that it enables a chief programmer or manager to exert real technical control over his teams, without delving into the morass of technical detail with which his programmers are often tempted to overwhelm him.

The solution to this problem is for the ambitious senior programmers of the present day to make the effort now to gain the necessary mastery of the subject, and so ensure that they will become in future the effective chief programmers, technical managers, and technical directors of their companies and institutions.

One way of acquiring a professional reorientation of this kind is to take a specialist postgraduate post-experience course in a new and important subject. Thus an electronic engineer might now be going back to university to study VLSI design: or an industrial chemist might be taking a Master's course in polymer science or genetic engineering, offered by some forward-looking university or polytechnic. I believe that ambitious programmers should not be reluctant to follow the example of the well-established engineering disciplines. That is why at Oxford University we have instituted a new MSc course in computation, devoted primarily to the objective of improving programming methods and ensuring their wider application. A similar course is offered at the Wang Institute in the USA.

## 18.7 Conclusion

In 1828, on the occasion of the grant of Royal Charter to the Institution of Civil Engineers, Thomas Tredgold defined civil engineering as 'the art of directing the great sources of power in Nature for the use and convenience of man'. Many branches of engineering have been established since that

date. They have all been concerned with the capture, storage and transformation of energy, or with the processing, shaping and assembly of materials. Computer programmers work with neither energy nor materials, but with a more intangible concept. We are concerned with the capture, storage, and processing of information. When the nature of our activities is more widely understood, both within our profession and outside, then we shall be deservedly recognized and respected as a branch of engineering. And I believe that in our branch of engineering, above all others, the academic ideals of rigour and elegance will pay the highest dividends in practical terms of reducing costs, increasing performance, and in directing the great sources of computational power on the surface of a silicon chip to the use and convenience of man.

It has long been my personal view that the separation of practical and theoretical work is artificial and injurious. Much of the practical work done in computing, both in software and in hardware design, is unsound and clumsy because the people who do it do not have any clear understanding of the fundamental principles underlying their work. Most of the abstract mathematical and theoretical work is sterile because it has no point of contact with real computing. One of the central aims of the Programming Research Group as a teaching and research group has been to set up an atmosphere in which this separation cannot happen ...

Christopher Strachey (1974)